

Exam Solution for TDT4186

Spring 2026

1 Multi-Choice Questions

Automatic grading. Attention: Each question is worth 1.5 points.

1. To load and transfer control to the OS kernel after power-on
2. Microkernel
3. When `write()` is invoked — it is a system call that triggers a software interrupt (or `syscall` instruction), causing the CPU to switch to kernel mode
4. Running → Blocked (Waiting), then Ready → Running again after the I/O completes
5. `fork()` duplicates the parent's address space into a new process; `exec()` replaces that address space entirely with a new program image
6. child: `x = 99` and parent: `x = 10`. Each process has its own copy of the address space; the child's write does not affect the parent
7. The parent blocks until the child exits
8. Heap memory and open file descriptors
9. Creating a thread is cheaper than forking a process because threads share the address space and require no new page table or file descriptor table duplication
10. SJF improves turnaround time over FIFO, but a short job that arrives after a long job has already started must still wait for it to finish
11. RR improves response time but hurts turnaround time
12. P2. CFS always selects the process with the least virtual runtime, ensuring each process receives a proportionally equal share of the CPU over time

13. P1 receives twice as much actual CPU time as P2
14. Virtual memory gives each process its own private address space, so the same virtual address in two different processes maps to different physical locations
15. The address 0x1000 the programmer sees is a virtual address, so the faulty data does not reside at physical address 0x1000
16. The base register shifts every virtual address by a fixed offset to produce a physical address
17. The free memory is scattered across multiple non-contiguous holes
18. Segmentation allocates variable-size regions, leading to external fragmentation, while paging uses fixed-size pages so free memory never becomes unusably scattered
19. Copy-on-write
20. The offset identifies the exact byte location within a physical page
21. Four accesses
22. Normal pages use 12 offset bits and large pages use 21 offset bits
23. Demand paging is more suitable. Since access patterns are random, prefetching predictions will frequently be wrong, wasting memory and disk I/O on pages that are never accessed
24. 6
25. The OS swaps pages of inactive processes out to disk, freeing physical frames
26. B
27. Replace if with while so the condition is rechecked after the thread wakes up
28. Circular wait
29. The spinning thread wastes CPU cycles for 200ms doing no useful work
30. 0
31. Use polling for the network card and interrupts for the disks
32. Device registers are mapped into the physical address space

- 33. The CPU is free to execute other processes
- 34. Memory-mapped I/O for initiation, DMA for the transfer, and an interrupt for completion
- 35. fd1 and fd2 are two distinct file descriptors
- 36. The file's metadata (size, permissions, timestamps) and pointers to its data blocks, but not its filename
- 37. The path is removed from the directory and the reference count is decremented to 1
- 38. The inode bitmap tracks which inodes are free or allocated, and the data block bitmap tracks which data blocks are free or allocated
- 39. The old path is removed and a new path is created pointing to the same inode, while the inode remains untouched
- 40. The parent directory's inode (user)

2 Multi-part questions

For multi-part questions, the automatic grading may fail to award some answers which are slightly different from the provided solutions or written in different phrases. Thus, it is worth noting that all wrong blanks should go through a scrutiny check. Answers that provide a close meaning can be awarded the point without deduction.

Process

(1 point each)

- 1. Ready, Runnable
- 2. Running
- 3. Blocked
- 4. Scheduled
- 5. Descheduled, preempted (Blocked is not correct)

Scheduling

(2 points each)

- Q0
- Q1
- 10
- 10
- more

Paging

(2 points each)

- 20
- 10
- 48
- 345
- 3

Concurrency

(1 point each)

1. 0
2. 0
3. 1
4. sem_post(&even);
5. sem_post(&odd);
6. sem_wait(&odd);
7. sem_wait(&even);

File systems

(2 points each)

1. Root directory data block
2. home directory data block
3. user directory data block
4. report.txt inode